
Fasal AI — Multi-Agent Smart Farming System: Build Prompts

This document contains step-by-step prompts to build the Fasal AI agentic system. Each step is a self-contained prompt you give to the platform. Complete and verify each step before moving to the next.

Agents in This System

Agent	Responsibility	Key Tools
Orchestrator Agent	Routes input, synthesises all outputs	CrewAI, Claude
Vision Agent	Crop disease & pest identification	Claude Vision, Plant.id
Weather Agent	Live weather + proactive alerts	Open-Meteo API
Agronomy Agent	Fertilizer, pest control, Q&A, yield	RAG, Claude LLM
Export Compliance Agent	MRL, PSQCA, packaging checks	Standards KB
Language Agent	Urdu/English output + WhatsApp	Claude, Twilio

Step 1 — Create the Project Structure

Create a Python project called **fasal-ai** with this exact structure:

```
fasal-ai/
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── vision_agent.py
│   │   ├── weather_agent.py
│   │   ├── agronomy_agent.py
│   │   ├── export_compliance_agent.py
│   │   └── language_agent.py
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── weather_tool.py
│   │   └── plant_id_tool.py
```

```
| | └─ mrl_checker_tool.py
| └─ data/
| | └─ crop_knowledge.json
| | └─ mrl_database.json
| | └─ psqca_standards.json
| └─ main.py
└─ app.py

└─ .env

└─ .env.example

└─ requirements.txt

└─ README.md
```

requirements.txt must include:

crewai, openai, python-dotenv, streamlit, requests, pillow, chromadb, langchain

.env.example must contain:

OPENAI_API_KEY=your-key-here

ANTHROPIC_API_KEY=your-key-here

PLANT_ID_API_KEY=your-key-here

TWILIO_ACCOUNT_SID=your-sid-here

TWILIO_AUTH_TOKEN=your-token-here

TWILIO_WHATSAPP_FROM=whatsapp:+14155238886

In main.py:

- Load all keys from .env using python-dotenv
- Print "Setup complete. All keys loaded." if OPENAI_API_KEY and ANTHROPIC_API_KEY are found
- Print "ERROR: Missing API keys." with the names of missing keys if any are absent
- All agent files, tool files, and *init.py* should be empty for now
- data/ JSON files should contain empty dicts {} for now

Note: Verify by running `python src/main.py` and confirming the success message prints before moving to Step 2.

Step 2 — Build the Vision Agent: Crop Disease & Pest Detection

In `src/agents/vision_agent.py` write a function called `analyze_crop(image_input: dict) -> dict`.

`image_input` will always contain:

- **image_base64**: base64-encoded string of the crop image uploaded by the farmer
- **crop_type**: a string e.g. "wheat", "mango", "cotton"
- **location**: a string e.g. "Lahore, Punjab"
- **farmer_description**: a string e.g. "yellow spots on leaves, started 3 days ago"

The function should:

- Initialize the Anthropic client using the `ANTHROPIC_API_KEY` from `.env`
- Send a `messages.create` request to `claude-opus-4-5` with vision enabled, passing the base64 image
- Use a system prompt that says: *You are an expert agricultural plant pathologist for Pakistan. Analyze the crop image and return ONLY a raw JSON object with no markdown and no explanation with these exact keys:*
 - `disease_detected`: string name of disease or "healthy" or "pest_damage"
 - `confidence`: one of "high", "medium", "low"
 - `severity`: one of "mild", "moderate", "severe" or "none"
 - `affected_area_percent`: integer estimate of crop area affected
 - `disease_description`: 2 sentences describing what is visible
 - `immediate_action`: the single most urgent thing the farmer must do today
 - `cause`: one sentence explaining the root cause
 - `spread_risk`: one of "low", "medium", "high"
- Also call the Plant.id API (<https://api.plant.id/v2/identify>) as a secondary confirmation, passing the base64 image
- Merge the Plant.id result into the response under the key `plant_id_confirmation`: the top suggestion name and probability
- Strip any markdown from both responses, parse and return the merged JSON dict
- Wrap in `try/except` and raise a clear error if parsing or the API call fails
- Print "Vision analysis complete. Disease: [`disease_detected`] | Confidence: [`confidence`]" on success

Note: Do not call this function yet. Just define it cleanly. Verify the file is importable before moving on.

Step 3 — Build the Weather Agent: Live Data + Proactive Alerts

In `src/agents/weather_agent.py` write a function called `get_weather_advice(farm_context: dict) -> dict`.

farm_context will always contain:

- **location:** city name string e.g. "Multan"
- **crop_type:** string e.g. "wheat"
- **crop_stage:** string e.g. "flowering", "germination", "harvest-ready"
- **last_irrigation_date:** string e.g. "2025-05-10"

The function should:

- Call the Open-Meteo geocoding API (<https://geocoding-api.open-meteo.com/v1/search>) to convert the location string to latitude/longitude — no API key needed
- Call the Open-Meteo forecast API (<https://api.open-meteo.com/v1/forecast>) with the coordinates, requesting: `temperature_2m_max`, `temperature_2m_min`, `precipitation_sum`, `wind_speed_10m_max` for the next 7 days
- Pass the raw weather data to Claude `claude-opus-4-5` with this system prompt: *You are an agricultural weather advisor for Pakistani farmers. Based on the weather forecast and farm context provided, return ONLY a raw JSON object with no markdown and no explanation with these exact keys:*
 - `current_conditions`: 1 sentence summary of today's weather
 - `forecast_summary`: 2 sentences covering the next 7 days
 - `irrigation_recommendation`: "irrigate_today", "skip_today", "irrigate_in_N_days"
 - `irrigation_reason`: 1 sentence explaining why
 - `spray_window`: "good" or "avoid" — whether conditions suit pesticide spraying
 - `spray_reason`: 1 sentence explanation
 - `frost_risk`: true or false
 - `heat_stress_risk`: true or false
 - `proactive_alerts`: list of 0-3 alert strings the farmer must know unprompted
e.g. ["Heavy rain in 2 days — delay fertilizer application",
"Wind speeds above 30km/h on Thursday — avoid spraying"]
 - `weekly_plan`: list of 7 strings, one farming action per day based on forecast
 - Strip markdown, parse and return the JSON dict
 - Wrap in try/except and raise a clear error if any API call or parsing fails
 - Print "Weather analysis complete for [location]. [N] proactive alerts generated." on success

Note: Open-Meteo is completely free with no API key. Do not call this function yet. Just define it cleanly.

Step 4 — Build the Agronomy Agent: Fertilizer, Pest Control, Q&A, Yield

In `src/agents/agronomy_agent.py` write a function called `get_agronomy_advice(query_context: dict) -> dict`.

query_context will always contain:

- **crop_type:** string e.g. "wheat"
- **crop_stage:** string e.g. "tillering"
- **disease_result:** the full dict returned by `analyze_crop()` — may be None if no image was uploaded
- **weather_result:** the full dict returned by `get_weather_advice()`
- **farmer_question:** free-text string in Urdu or English e.g. "میری گندم کے پتے پیلے ہو رہے ہیں"
- **soil_type:** string e.g. "loamy", "clay", "sandy"
- **area_acres:** integer number of acres

The function should:

- Initialize the Anthropic client using `ANTHROPIC_API_KEY` from `.env`
- Build a RAG context string by loading `src/data/crop_knowledge.json` and retrieving the entry for `crop_type` — if not found, proceed without it
- Send a `messages.create` request to `claude-opus-4-5` with this system prompt: *You are a senior agricultural expert for Pakistan with deep knowledge of local crops, soils, and farming practices. Given the full context below, return ONLY a raw JSON object with no markdown and no explanation with these exact keys:*

- `fertilizer_recommendation`: object with keys:

- `type`: fertilizer name e.g. "Urea + DAP"
- `quantity_per_acre`: string e.g. "1 bag Urea + half bag DAP"
- `timing`: when to apply e.g. "Apply within 3 days before irrigation"
- `reason`: 1 sentence why

- `pest_control`: object with keys:

- `pest_identified`: string or "none"
- `treatment`: chemical or biological control method
- `dosage`: specific quantity and dilution
- `safety_note`: precaution for the farmer

- `direct_answer`: 2-3 sentence plain-language answer to `farmer_question`
- `yield_prediction`: object with keys:
 - `expected_yield_per_acre`: string e.g. "35-40 maunds"
 - `confidence`: "high", "medium", "low"
 - `risk_factors`: list of 2-3 things that could reduce yield
- `irrigation_schedule`: list of 7 objects, one per day, each with:
 - `day`: "Monday" etc.
 - `action`: "irrigate" or "skip"
 - `duration_hours`: integer or 0
 - `reason`: short string
- `weekly_action_plan`: list of 5 prioritized action strings for this week
 - Include the full `disease_result`, `weather_result`, RAG context, and `farmer_question` in the user message
 - Strip markdown, parse and return the JSON dict
 - Wrap in try/except and raise a clear error if parsing fails
 - Print "Agronomy advice complete for [crop_type] — [area_acres] acres." on success

Note: Do not call this function yet. The `crop_knowledge.json` file can remain as {} for now — the agent will still work using Claude's built-in knowledge.

Step 5 — Build the Export Compliance Agent: MRL, PSQCA, Packaging

In `src/agents/export_compliance_agent.py` write a function called `check_export_compliance(export_context: dict) -> dict`.

`export_context` will always contain:

- **crop_type**: string e.g. "mango"
- **variety**: string e.g. "Sindhri", "Chaunsa"
- **target_country**: string e.g. "UAE", "UK", "Germany", "Saudi Arabia", "China", "USA"
- **pesticides_used**: list of strings e.g. ["Chlorpyrifos", "Imidacloprid"]
- **fertilizers_used**: list of strings e.g. ["Urea", "DAP"]
- **harvest_date**: string e.g. "2025-06-15"
- **quantity_kg**: integer

The function should:

- Initialize the Anthropic client using `ANTHROPIC_API_KEY` from `.env`

- Load src/data/mrl_database.json — if empty, proceed using Claude's knowledge of EU Regulation 396/2005, UAE GSO standards, UK FSA, USDA, and Codex Alimentarius
- Load src/data/psqca_standards.json — if empty, proceed using Claude's knowledge of PSQCA grading standards
- Send a messages.create request to claude-opus-4-5 with this system prompt: *You are a Pakistan agricultural export compliance expert with full knowledge of PSQCA standards, international MRL regulations, phytosanitary requirements, and packaging rules for Pakistan's top export markets. Return ONLY a raw JSON object with no markdown and no explanation with these exact keys:*
 - overall_status: one of "READY_TO_EXPORT", "ACTION_REQUIRED", "EXPORT_BLOCKED"
 - compliance_checks: list of check objects, one per check category, each with:
 - category: string e.g. "Pesticide MRL", "PSQCA Grade", "Packaging"
 - status: one of "PASS", "WARNING", "FAIL"
 - details: 1-2 sentences explaining the result
 - action_required: string or null — what the farmer must do to fix this
 - banned_substances: list of any pesticides or fertilizers that are banned in the target country — empty list if none found
 - phytosanitary_certificate: object with keys:
 - required: true or false
 - issuing_authority: string e.g. "DPPRD Karachi"
 - contact: phone or address string
 - days_before_shipment: integer
 - packaging_requirements: object with keys:
 - label_language: list of required languages e.g. ["Arabic", "English"]
 - country_of_origin_required: true or false
 - weight_marking: required format string
 - cold_chain_temp_celsius: integer or null
 - certifications_that_add_value: list of certification name strings e.g. ["GlobalGAP", "Halal", "Organic"]
 - summary: 3 sentences plain-language summary for the farmer
 - Strip markdown, parse and return the JSON dict
 - Wrap in try/except and raise a clear error if parsing fails
 - Print "Export compliance check complete. Status: [overall_status]" on success

Note: Do not call this function yet. Just define it cleanly. This agent works entirely from Claude's knowledge of international trade standards.

Step 6 — Build the Language & Output Agent: Urdu/English + WhatsApp

In `src/agents/language_agent.py` write a function called `format_final_output(all_results: dict) -> dict`.

all_results will always contain:

- **vision_result:** dict from `analyze_crop()` or None
- **weather_result:** dict from `get_weather_advice()`
- **agronomy_result:** dict from `get_agronomy_advice()`
- **export_result:** dict from `check_export_compliance()` or None
- **output_language:** "urdu" or "english"
- **farmer_phone:** WhatsApp number string e.g. "+923001234567" or None

The function should:

- Initialize the Anthropic client using `ANTHROPIC_API_KEY` from `.env`
- Send a `messages.create` request to `claude-opus-4-5` with this system prompt: *You are a bilingual agricultural communication expert. Your job is to take expert technical advice and rewrite it in simple, clear language for a Pakistani farmer with limited education. Return ONLY a raw JSON object with no markdown and no explanation with these exact keys:*
 - `greeting`: a warm 1-sentence greeting in the chosen language
 - `disease_summary`: 2 sentences about the crop diagnosis in plain language
(omit section if `vision_result` is None)
 - `weather_summary`: 2 sentences about today's weather and what it means
 - `top_actions`: list of exactly 5 prioritized action strings, numbered, in plain simple language. Start each with a verb.
e.g. "1. Spray Dimethoate today before 9am"
 - `irrigation_today`: single sentence — irrigate or skip and why
 - `export_status`: 1 sentence summary of export readiness
(omit if `export_result` is None)
 - `whatsapp_message`: a short WhatsApp-friendly version of the advice, under 300 characters, starting with "Fasal AI:"

- If `output_language` is "urdu", all text values in the JSON must be written in Urdu script. Keys remain in English.
- After generating the JSON, if `farmer_phone` is not None, send `whatsapp_message` via Twilio WhatsApp API using TWILIO credentials from `.env`
- Strip markdown, parse and return the JSON dict along with a `sent_whatsapp: true/false` key
- Wrap in try/except — if Twilio fails, still return the JSON with `sent_whatsapp: false`, do not raise
- Print "Output formatted in [language]. WhatsApp sent: [true/false]" on success

Note: Do not call this function yet. Twilio WhatsApp has a sandbox you can use for free — check their docs to activate it.

Step 7 — Wire All Agents into the Orchestrator Pipeline in `main.py`

In `src/main.py` wire all five agents into a single Orchestrator pipeline.

The file should:

- Load all keys from `.env` using `python-dotenv` and raise `EnvironmentError` if `OPENAI_API_KEY` or `ANTHROPIC_API_KEY` is missing
- Import `analyze_crop`, `get_weather_advice`, `get_agronomy_advice`, `check_export_compliance`, `format_final_output`

Define a hardcoded test scenario:

```
test_scenario = {
    "crop_type": "wheat",
    "variety": "Punjab-11",
    "location": "Lahore, Punjab",
    "crop_stage": "tillering",
    "soil_type": "loamy",
    "area_acres": 5,
    "last_irrigation_date": "2025-05-08",
    "farmer_question": "My wheat leaves are turning yellow, what should I do?",
    "farmer_phone": None,
    "output_language": "english",
    "image_base64": None,
    "farmer_description": "Yellow spots appearing on lower leaves, started 5 days ago",
}
```

```

"target_country": "UAE",

"pesticides_used": ["Chlorpyrifos", "Imidacloprid"],

"fertilizers_used": ["Urea", "DAP"],

"harvest_date": "2025-06-20",

"quantity_kg": 5000

}

```

Run the full pipeline in order:

- If image_base64 is not None: call analyze_crop — print disease_detected and confidence
- Call get_weather_advice — print current_conditions and number of proactive_alerts
- Call get_agronomy_advice with disease_result, weather_result, and farmer_question — print fertilizer type and yield prediction
- If target_country is provided: call check_export_compliance — print overall_status
- Call format_final_output with all results — print top_actions as a numbered list
- Wrap the entire pipeline in try/except and print a clear error message if anything fails
- Print "Fasal AI pipeline complete." at the end if everything succeeds

Note: Run python src/main.py and confirm all agents fire in sequence and outputs print correctly before moving to Step 8.

Step 8 — Build the Streamlit UI in app.py

In **src/app.py** build the full Fasal AI Streamlit UI. Do not change any agent files.

Page setup:

- Page config: title "Fasal AI — Smart Farming Assistant", layout "wide", page_icon "🌾"
- Two-column layout: left column (width 2) for inputs, right column (width 3) for results
- Custom CSS: green primary color (#1D9E75), clean white cards, mobile-friendly font sizes

Left column — Input form:

- A logo area: show "Fasal AI" in large green text with tagline "آپ کے کھیت کا ذہین ساتھی"
- st.file_uploader: Accept jpg/png — label "Upload crop image (optional)"
- st.text_input: "Crop type" with placeholder "e.g. wheat, mango, cotton, sugarcane"
- st.text_input: "Your location" with placeholder "e.g. Lahore, Multan, Karachi"
- st.selectbox: "Crop stage" options: Germination, Tillering, Flowering, Grain-filling, Harvest-ready

- st.selectbox: "Soil type" options: Loamy, Clay, Sandy, Silty
- st.number_input: "Farm size (acres)" min 1 max 5000 default 5
- st.text_area: "Describe the problem" placeholder "Describe what you see on your crop..."
- st.text_input: "Last irrigation date" placeholder "YYYY-MM-DD"
- st.radio: "Output language" options: English, اردو
- st.text_input: "WhatsApp number (optional)" placeholder "+923001234567"

Export compliance section (expandable):

- st.expander: "Export compliance check (optional)"
- Inside: target country selectbox (UAE, Saudi Arabia, EU, UK, China, USA)
- Inside: multiselect for pesticides used (common Pakistani pesticides list)
- Inside: text input for harvest date
- Inside: number input for quantity in kg

Primary button "Analyze My Crop" — when clicked:

- Validate that crop_type, location, and crop_stage are filled — show st.error and stop if empty
- If image was uploaded, convert it to base64 string for the Vision Agent
- Run all applicable agents in sequence, each inside its own st.spinner:

Spinner 1: "Analyzing crop image..." — only if image uploaded

Spinner 2: "Checking weather for [location]..."

Spinner 3: "Getting expert farming advice..."

Spinner 4: "Checking export compliance..." — only if country selected

Spinner 5: "Preparing your personalized report..."

- Store the final output in st.session_state['fasal_result']

Right column — Results display (show only after pipeline runs):

- st.success: show the greeting from language_agent output
- Section 1 — Disease Diagnosis (if image was uploaded): st.metric cards for disease name, severity, confidence, affected area %; st.info box for disease_description and immediate_action
- Section 2 — Weather & Irrigation: st.info for current_conditions; st.warning for each proactive_alert; st.expander "7-day irrigation schedule" with table
- Section 3 — Your Action Plan (most prominent): st.subheader "Your Action Plan This Week"; each action in top_actions as numbered st.info box with green left border

- Section 4 — Fertilizer & Pest Control: two side-by-side cards; fertilizer on left (type, quantity_per_acre, timing); pest on right (treatment, dosage, safety_note)
- Section 5 — Export Compliance Report (if run): overall status badge (green/amber/red); each compliance check with icon, category, details, action_required; phytosanitary certificate in st.info; packaging requirements in table
- Bottom: WhatsApp preview box showing whatsapp_message
- If farmer_phone provided and sent_whatsapp is true: st.success "WhatsApp sent!"
- "Analyze Another Crop" button that clears session state and reruns
- Wrap all agent calls in try/except and show st.error if anything fails

Note: Run with streamlit run src/app.py — Verify all agents fire, all sections display, export compliance appears when a country is selected, and WhatsApp preview shows at the bottom.

Deployment & Demo Tips

- Deploy the Streamlit app to Streamlit Cloud (free) for a live URL to show judges
- For the demo: pre-load a real wheat or mango disease image to show Vision Agent working
- Set a Pakistani city as location so Open-Meteo returns real weather data live
- Show the agent spinner sequence on screen — this proves agentic tool-calling to judges
- Enable export compliance with UAE as target and Chlorpyrifos in pesticides — will generate realistic warnings
- Switch output language to Urdu mid-demo to show localization capability